

1.1 Numpy中的数组

1.1.1 Numpy的数据格式

numpy中所支持的数据类型其实相比原生Python，它更接近于C。光是整数类型，就分了int32, int64, uint32, uint64等好几个类型。包括浮点数也分为float32和float64。在字长和类型上进行了更好的统一，方便numpy与C的无缝衔接。

而我们之所以引入Numpy，一个很重要的原因就是Numpy数组可以很好地模拟向量与矩阵运算。我们可以通过numpy.array创建一个数组，例如：

```
import numpy as np
x=np.array([1,2,3,4,5])
```

这样我们可以创建一个numpy数组。但是numpy的数组和Python原生的列表数据结构并不相同，它不能直接append、pop等。并且在numpy数组中，元素的数据类型必须要统一，不允许多个不同类型数据存放在同一个数组当中。那numpy数组具备怎样的特性呢？在后面我们会看到，numpy.array很重要的特性就是它可以模拟线性代数当中的各种运算。

与二重列表类似，numpy也支持二重数组以及更高维度的多重数组。并且numpy约定，每个子数组的长度必须保持平齐。

那我们如何指定一个数组的数据类型呢？我们可以通过这样的方法：

```
x=np.array([1,2,3],dtype=np.int32)
```

这样，我们创建了一个数据类型为numpy.int32类型的数组。

numpy数组与其他数据类型之间也可以进行方便的数据转换。例如，列表、元组可以转换为numpy.array，而numpy.array也可以转换为列表等数据结构。例如，我们有两种方法把上述数组修改为Python列表类型：

```
# 方法1：直接强制类型转换：
x=list(x)
# 方法2：使用tolist方法
x=x.tolist()
```

那么如果想要修改numpy数组当中的数据元素类型，我们可以通过astype指定。

```
x.astype(np.float32)
```

在numpy当中，有一些快速创建数组的方法，例如：

1. `numpy.zeros((m,n))`

这个方法被用于创建一个形状为 (m, n) 的全0数组。例如：

案例：

```
import numpy as np

# 创建一个2x3的零矩阵
zero_matrix = np.zeros((2, 3))
print(zero_matrix)

# 输出：
# [[0. 0. 0.]
#  [0. 0. 0.]]
```

2. `numpy.ones((m,n))`

这个方法被用于创建一个形状为 (m, n) 的全1数组。例如：

案例：

```
import numpy as np

# 创建一个2x3的全1矩阵
ones_matrix = np.ones((2, 3))
print(ones_matrix)

# 输出：
# [[1. 1. 1.]
#  [1. 1. 1.]]
```

注意，这两个方法传进去的参数都是元组！要记得打两次括号！当然，如果只传入一个数字也是可以的，它会默认给你创建一个n*n的方阵。

3. `numpy.full((m,n), value)`

创建一个形状为 (m, n) 的数组，并用给定的值 value 填充数组中的所有元素。

案例：

```
import numpy as np

# 创建一个2x3的数组，所有元素都是42
full_matrix = np.full((2, 3), 42)
print(full_matrix)

# 输出：
# [[42 42 42]
#  [42 42 42]]
```

4. numpy.arange(start, stop, step)

这个方法可以生成一个等差数列，从 start （包括）到 stop （不包括），步长为 step 。如果省略 step ，则默认为1。

案例：

```
import numpy as np

# 生成从0到5（不包括5）的数组，步长为1
arange_array = np.arange(0, 5)
print(arange_array)

# 输出：
# [0 1 2 3 4]

# 生成从0到10的数组，步长为2
arange_step_array = np.arange(0, 11, 2)
print(arange_step_array)

# 输出：
# [ 0  2  4  6  8 10]
```

它和Python自带的range函数用法类似，但是np.arange返回的是一个实打实的向量，而Python自带的range嘛.....严格意义上来讲，应该叫做一个迭代器。

5. numpy.linspace(start, stop, num)

在指定的间隔内返回均匀间隔的数字，即生成一个等差数列，但这里是基于元素数量 num 来均匀分配间隔。

案例：

```
import numpy as np

# 生成从0到10的5个等间隔的数
linspace_array = np.linspace(0, 10, 5)
print(linspace_array)

# 输出:
# [ 0.   2.5  5.   7.5 10. ]
```

6. `numpy.diag(array, k=0)`

如果 `array` 是一维数组，则返回一个以 `array` 为对角线的方阵（`k=0` 表示主对角线）。如果 `array` 是二维数组，则返回其指定对角线上的元素。

案例：

```
import numpy as np

# 使用一维数组创建对角矩阵
diag_matrix = np.diag([1, 2, 3])
print(diag_matrix)

# 输出:
# [[1 0 0]
#  [0 2 0]
#  [0 0 3]]

# 使用二维数组提取对角线元素
diag_elements = np.diag(np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]]))
print(diag_elements)

# 输出:
# [1 5 9]
```

7. `numpy.eye(N, M=None, k=0, dtype=float, order='C')`

这个方法返回一个二维数组（矩阵），其中对角线元素为1，其余元素为0。这也正是单位矩阵的定义。如果 `M` 为 `None`，则与 `N` 相同，即生成一个 `N x N` 的单位矩阵。`k` 指定了对角线的索引，`k=0` 表示主对角线。

这里是 `numpy.eye()` 的完整案例：

```
import numpy as np

# 创建一个3x3的单位矩阵
eye_matrix = np.eye(3)
print(eye_matrix)
# 输出:
# [[1. 0. 0.]
#  [0. 1. 0.]
#  [0. 0. 1.]]

# 创建一个3x4的矩阵，其主对角线元素为1，其余为0
eye_matrix_3x4 = np.eye(3, 4)
print(eye_matrix_3x4)
# 输出:
# [[1. 0. 0. 0.]
#  [0. 1. 0. 0.]
#  [0. 0. 1. 0.]]

# 创建一个3x3的矩阵，其对角线向上偏移一个单位（次对角线为1）
eye_matrix_k1 = np.eye(3, k=1)
print(eye_matrix_k1)
# 输出:
# [[0. 1. 0.]
#  [0. 0. 1.]
#  [0. 0. 0.]]

# 创建一个3x3的矩阵，其对角线向下偏移一个单位（超对角线为1）
eye_matrix_k_1 = np.eye(3, k=-1)
print(eye_matrix_k_1)
# 输出:
# [[0. 0. 0.]
#  [1. 0. 0.]
#  [0. 1. 0.]]
```

在这个案例中，`np.eye(3)` 生成了一个3x3的单位矩阵，其中主对角线上的元素为1，其余元素为0。`np.eye(3, 4)` 创建了一个3x4的矩阵，其形状不是方阵，但主对角线上的元素仍然为1。通过 `k` 参数，你可以控制对角线的位置，`k=1` 表示次对角线（对角线向上偏移一个单位）为1，而 `k=-1` 表示超对角线（对角线向下偏移一个单位）为1。

通过这些方法，可以快速创建numpy数组。

1.1.2 Numpy数组的变换

1. 数组的转置

数组（或者说矩阵）的转置是指将数组的行和列进行互换。对于二维数组（矩阵）来说，转置意味着将原矩阵的行列互换，即原矩阵的第*i*行第*j*列的元素会变为转置矩阵的第*j*行第*i*列的元素。NumPy提供了非常简便的方法来实现数组的转置，主要通过 `.T` 属性或者 `numpy.transpose()` 函数。使用 `.T` 属性这是最常用的方法，因为它既简洁又直观。

```
import numpy as np

# 创建一个二维数组
arr = np.array([[1, 2, 3],
                [4, 5, 6],
                [7, 8, 9]])

# 使用.T属性进行转置
arr_T = arr.T

print(arr_T)
```

输出将会是：

```
[[1 4 7]
 [2 5 8]
 [3 6 9]]
```

`numpy.transpose()` 函数也可以用来转置数组，其用法与 `.T` 属性类似，但提供了更多的灵活性，比如可以指定转置的轴。

```
# 使用numpy.transpose()函数进行转置
arr_transposed = np.transpose(arr)

print(arr_transposed)
```

输出与上面使用 `.T` 属性的例子相同。

在更复杂的数组中（比如三维或更高维度的数组），`.T` 属性只会交换前两个轴，而 `numpy.transpose()` 函数允许你通过 `axes` 参数来指定任意轴的转置顺序。例如，在三维数组中，你可

以通过指定 `axes` 参数来定义三个轴之间的转置关系。

另外，向量也是可以进行转置的。如果你只用一层中括号表示向量，那么它转置与否无所谓。但如果是用两层括号表示向量，也就是二维数组的形式，那么向量就会有行向量和列向量之分。

2. 数组的变形

在NumPy中，改变数组的形状通常是通过 `reshape` 函数来实现的。`reshape` 函数允许你在不改变数组数据的情况下，重新排列数组的元素以形成新的形状。这要求新形状的总元素数量与原始数组的总元素数量相同。

`reshape` 函数的基本语法如下：

```
numpy.reshape(a, newshape)
```

- `a` 是要改变的原始数组。
- `newshape` 是新的形状，可以是整数或整数元组的形式。

如果 `newshape` 是一个整数，那么它会被视为新的形状中的一维长度，NumPy会自动计算出其他维度的长度。如果 `newshape` 是整数元组，那么它应该明确指定每个维度的大小。

假设我们有一个一维数组，我们想要将其重塑为一个二维数组：

```
import numpy as np

# 创建一个一维数组
arr = np.arange(6) # array([0, 1, 2, 3, 4, 5])

# 使用reshape重塑为2x3的二维数组
arr_resaped = arr.reshape((2, 3))

print(arr_resaped)

# 输出：
# [[0 1 2]
#  [3 4 5]]
```

在这个例子中，`reshape` 将原始的一维数组 `arr` 重塑为一个2行3列的二维数组 `arr_resaped`。

这里需要注意两点：

- 原始数组的总元素数量必须与新形状的总元素数量相匹配，否则 `reshape` 会抛出异常。

- 使用 -1 作为 newshape 中的一个维度可以让NumPy自动计算该维度的大小。这在你不知道某个维度具体大小但知道其他维度大小时非常有用。

例如，如果你有一个包含6个元素的一维数组，并且你想要重塑为一个3行的二维数组，但不确定每行应该有多少个元素，你可以这样做：

```
arr_resaped = arr.reshape((3, -1))
print(arr_resaped)
# 输出：
# [[0 1]
#  [2 3]
#  [4 5]]
```

在这个例子中，NumPy自动计算出第二维（列）应该是2，以便总元素数量保持为6。

另外，reshape 函数返回一个新的数组对象，该对象具有指定的形状，但包含与原始数组相同的数据。原始数组不会被修改。

当然，当你想要将一个多维数组转换为一维数组时，np.flatten() 是一个非常有用的函数。np.flatten() 会返回数组的一份拷贝，该拷贝被转换成了一维数组，且保持原始数据的顺序不变。

```
numpy.flatten(a, order='C')
```

```
import numpy as np
```

```
# 创建一个二维数组
```

```
arr = np.array([[1, 2, 3], [4, 5, 6]])
```

```
# 使用flatten转换为一维数组
```

```
arr_flat = arr.flatten() # 注意：这里也可以直接使用 np.flatten(arr)，但使用数组对象的方法更常见
```

```
print(arr_flat)
```

```
# 输出：
```

```
# [1 2 3 4 5 6]
```

在这个例子中，flatten() 方法将二维数组 arr 转换成了一个 一维数组 arr_flat。

虽然 flatten() 可以很方便地将多维数组转换为一维数组，但使用 reshape(-1) 也是达到同样目的的一种常见且高效的方法。reshape(-1) 通过指定 -1 来自动计算新形状中唯一维度的大小，这通常用于

将数组展平。

```
arr_resaped = arr.reshape(-1)
print(arr_resaped)
# 输出与 arr_flat 相同
# [1 2 3 4 5 6]
```

在大多数情况下，`reshape(-1)` 和 `flatten()` 在功能上是等价的，但 `reshape(-1)` 可能在性能上更优，因为它不需要创建数组的额外副本。在需要明确保持数组连续性的场景中，使用 `reshape(-1)` 可能更为合适。

3. 数组的堆叠

在NumPy中，实现数组拼接主要有以下几种方法：

1. `numpy.concatenate()`：

这是最常用的数组拼接函数之一，它可以沿着指定的轴将多个数组序列连接起来。默认情况下，`concatenate` 函数会沿着第一个轴（`axis=0`）进行拼接，但你可以通过 `axis` 参数来指定其他轴。

```
import numpy as np

a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6]])

# 默认沿着第一个轴（axis=0）拼接
c = np.concatenate((a, b), axis=0)
print(c)
# 输出：
# [[1 2]
#  [3 4]
#  [5 6]]

# 沿着第二个轴（axis=1）拼接
d = np.concatenate((a, b.T), axis=1) # 注意b需要转置以匹配a的形状
print(d)
# 输出：
# [[1 2 5]
#  [3 4 6]]
```

2. `numpy.stack()`：

`stack` 函数将一系列数组沿着新的轴进行堆叠。这与 `concatenate` 不同，因为 `stack` 会创建一个新

的维度。

```
a = np.array([1, 2])
b = np.array([3, 4])

# 沿着新的轴 (axis=0) 堆叠
c = np.stack((a, b), axis=0)
print(c)
# 输出:
# [[1 2]
#  [3 4]]

# 沿着新的轴 (axis=1) 堆叠
d = np.stack((a, b), axis=1)
print(d)
# 输出:
# [[1 3]
#  [2 4]]
```

3. `numpy.vstack()` 和 `numpy.hstack()` :

这两个函数是 `concatenate` 的便捷包装, 分别用于垂直 (沿着第一个轴) 和水平 (沿着第二个轴) 堆叠数组。

- `vstack` (垂直堆叠) :

```
a = np.array([1, 2])
b = np.array([3, 4])
c = np.vstack((a, b))
print(c)
# 输出:
# [[1 2]
#  [3 4]]
```

- `hstack` (水平堆叠) :

```
a = np.array([[1], [2]])
b = np.array([[3], [4]])
c = np.hstack((a, b))
print(c)
# 输出:
# [[1 3]
#  [2 4]]
```

4. `numpy.column_stack()` 和 `numpy.row_stack()` :

这两个函数与 `hstack` 和 `vstack` 类似，但它们接受一维数组作为输入，并将它们视为列或行。

- `column_stack` (列堆叠) :

```
a = np.array([1, 2])
b = np.array([3, 4])
c = np.column_stack((a, b))
print(c)
# 输出:
# [[1 3]
#  [2 4]]
```

- `row_stack` (行堆叠) :

```
a = np.array([1, 2])
b = np.array([3, 4])
c = np.row_stack((a, b))
print(c)
# 输出:
# [[1 2]
#  [3 4]]
```

注意: `row_stack` 实际上就是 `vstack` 的别名。

在选择这些方法时，重要的是要考虑你的具体需求，比如你希望沿着哪个轴进行拼接，以及你的输入数组的形状。

4. 元素追加与删除

在NumPy中，数组（尤其是`ndarray`对象）通常被设计为固定大小的，这意呀着它们不像Python的列表（`list`）那样可以方便地追加或删除元素。然而，NumPy提供了一些方法来间接实现这些操作，但通常这些方法会涉及到创建新的数组，而不是在原地修改原始数组。

5. 追加元素

要在NumPy数组中追加元素，你可以使

用 `numpy.concatenate()`、`numpy.append()` 或 `numpy.vstack()`（对于二维数组作为行追加）等函数。不过，需要注意的是，`numpy.append()` 实际上是一个包装了 `numpy.concatenate()` 的便捷函数，但它通常比直接使用 `concatenate()` 要慢，因为它会创建更多的临时数组。

- 使用 `numpy.concatenate()`

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([4, 5])

# 沿着第一个轴 (axis=0) 追加
c = np.concatenate((a, b))
print(c) # 输出: [1 2 3 4 5]
```

- 使用 `numpy.append()`

```
a = np.array([1, 2, 3])

# 追加单个元素
b = np.append(a, 4)
print(b) # 输出: [1 2 3 4]

# 追加数组
c = np.append(a, [4, 5])
print(c) # 输出: [1 2 3 4 5]
```

在NumPy中删除元素通常意味着创建一个新的数组，它包含了除了要删除的元素之外的所有元素。这可以通过布尔索引、切片或 `numpy.delete()` 函数来实现。

- 使用布尔索引

```
a = np.array([1, 2, 3, 4, 5])

# 删除值为3的元素
mask = a != 3
b = a[mask]
print(b) # 输出: [1 2 4 5]
```

- 切片可以用来跳过特定的元素或元素范围，但这种方法不如布尔索引灵活。

```
a = np.array([1, 2, 3, 4, 5])

# 跳过第三个元素
b = np.concatenate((a[:2], a[3:]))
print(b) # 输出: [1 2 4 5]
```

- `numpy.delete()` 函数可以从数组中删除指定位置的元素。

```
a = np.array([1, 2, 3, 4, 5])
```

```
# 删除索引为2的元素（即值为3的元素）
```

```
b = np.delete(a, 2)
```

```
print(b) # 输出: [1 2 4 5]
```

注意

- 这些操作都会返回一个新的数组，而不是修改原始数组。
- 对于大型数组，频繁地创建新数组（尤其是在循环中）可能会导致性能问题。在这些情况下，考虑是否可以使用更高效的算法或数据结构，或者是否可以将数据转换为更适合追加和删除操作的形式（如Python列表）。
- `numpy.append()` 和 `numpy.delete()` 虽然方便，但在处理大数据集时可能不是最高效的方法。在这些情况下，考虑使用布尔索引或切片等更底层的方法。

1.1.3 数组广播机制

广播机制是NumPy在算术运算中处理不同形状数组的一种方式，它允许在不进行显式复制数据的情况下执行元素级别的操作，从而提高了代码的执行效率和可读性。当两个或多个数组的形状不一致时，NumPy会尝试通过广播机制将它们“扩展”到相同的形状，以便可以进行逐元素的算术运算。

在NumPy中，当两个或多个数组的形状不一致，但满足广播规则时，就会发生数组广播。具体来说，如果两个数组在维度上不匹配，但满足以下条件之一，就可以通过广播机制进行运算：

1. **维度较小的数组会在前面补1**，使得两个数组的维度数相同。
2. **在某个维度上，如果一个数组的大小为1**，则它会被视为在该维度上重复扩展，以匹配另一个数组的大小。
3. **如果两个数组在所有维度上都不匹配，并且没有任何一个维度的大小为1**，则无法进行广播，并会引发错误。

案例一：简单的一维数组广播

```
import numpy as np

A = np.array([1, 2, 3]) # 形状 (3,)
B = np.array([10, 20, 30, 40]) # 形状 (4,)

# 尝试进行加法运算，但直接相加会报错，因为形状不匹配
# 但如果我们将A视为 (1, 3) 形状的数组，并将B视为 (4, 1) 形状的数组，然后进行逐元素加法，则可以实现广播
# 然而，NumPy不会直接显示地改变数组的形状，而是隐式地进行广播

# 实际上，NumPy不允许直接将形状为 (3,) 和 (4,) 的数组进行广播
# 但如果我们将B的前三个元素与A的元素相加，可以模拟广播的效果（注意：这不是NumPy的默认行为）
# 为了演示广播，我们可以使用形状兼容的数组
C = np.array([10, 20, 30]) # 形状 (3,)
D = A + C # 这里可以广播，因为形状相同
print(D) # 输出: [11 22 33]

# 注意：上面的C和D的示例是为了说明广播的概念，而不是NumPy直接允许的操作
```

注意：上面的示例中，直接使用A和B进行广播是无效的，因为它们的形状在所有维度上都不匹配，且没有任何一个维度的大小为1。但为了说明广播的概念，我使用了形状兼容的数组C来与A进行加法运算。

案例二：二维数组与一维数组的广播

```
import numpy as np

A = np.array([[1, 2, 3], [4, 5, 6]]) # 形状 (2, 3)
B = np.array([10, 20, 30]) # 形状 (3,)

# B会被视为形状为 (1, 3) 的数组，然后在第一个维度上复制以匹配A的形状 (2, 3)
# 现在A和B具有相同的“广播后”形状，可以进行逐元素加法
C = A + B
print(C)
# 输出：
# [[11 22 33]
#  [14 25 36]]
```

在这个例子中，B数组（形状为 (3,)）在第一个维度上被隐式地扩展为 (1, 3)，然后在这个维度上复制以匹配A数组的形状 (2, 3)。这样，A和B就可以进行逐元素的加法运算了。

通过这两个案例，我们可以看到NumPy的广播机制是如何工作的，以及它如何允许我们在不显式改变数组形状的情况下进行高效的数组运算。

1.1.4 随机数生成与抽样

NumPy是Python中用于科学计算的一个基础库，它提供了大量的数学函数工具，特别是针对数组的操作。在NumPy中，生成随机数数组是一个常见的需求，NumPy提供了多种函数来满足这一需求。以下是NumPy中几种常用的随机数数组生成方法及其使用方式：

1. numpy.random.rand()

功能：生成指定形状的数组，数组中的元素在区间[0, 1)内均匀分布。

使用示例：

```
import numpy as np

# 生成一个具有5个元素的一维数组
arr1 = np.random.rand(5)
print(arr1)

# 生成一个2行3列的二维数组
arr2 = np.random.rand(2, 3)
print(arr2)
```

2. numpy.random.randn()

功能：生成指定形状的数组，数组中的元素服从标准正态分布（均值为0，标准差为1）。

使用示例：

```
import numpy as np

# 生成一个具有5个元素的一维数组
arr1 = np.random.randn(5)
print(arr1)

# 生成一个2行3列的二维数组
arr2 = np.random.randn(2, 3)
print(arr2)
```

3. numpy.random.randint()

功能：生成指定范围内的随机整数数组。可以指定范围的最小值、最大值和数组的形状。

使用示例：

```
import numpy as np

# 生成一个在[1, 10)之间的一维整数数组
arr1 = np.random.randint(1, 10, size=5)
print(arr1)

# 生成一个在[1, 10)之间2行3列的二维整数数组
arr2 = np.random.randint(1, 10, size=(2, 3))
print(arr2)
```

4. numpy.random.uniform()

功能：生成指定范围内的均匀分布的随机数数组。需要指定生成随机数的下界、上界和数组的形状。

使用示例：

```
import numpy as np

# 生成一个在[1, 5)之间的一维数组
arr1 = np.random.uniform(1, 5, size=5)
print(arr1)

# 生成一个在[1, 5)之间2行3列的二维数组
arr2 = np.random.uniform(1, 5, size=(2, 3))
print(arr2)
```

5. numpy.random.normal()

功能：生成指定均值和标准差的正态分布的随机数数组。需要指定生成随机数的均值、标准差和数组的形状。

使用示例：

```
import numpy as np

# 生成均值为2，标准差为0.5的一维数组
arr1 = np.random.normal(2, 0.5, size=5)
print(arr1)

# 生成均值为2，标准差为0.5的2行3列的二维数组
arr2 = np.random.normal(2, 0.5, size=(2, 3))
print(arr2)
```

6. numpy.random.random()

功能：与 `numpy.random.rand()` 类似，生成指定形状的数组，数组中的元素在区间[0, 1)内均匀分布。该函数是Python标准库 `random` 模块的函数的向量化版本。

使用示例：

```
import numpy as np

# 生成一个3x3的二维数组
arr = np.random.random((3, 3))
print(arr)
```

7. numpy.random.seed()

功能：在生成伪随机数时指定种子。指定相同的种子将会产生相同的随机数序列，这在调试代码时非常有用。

使用示例：

```
import numpy as np

np.random.seed(0)
arr1 = np.random.rand(3, 3)
print(arr1) # 每次运行这段代码时，如果seed相同，生成的随机数序列也将相同
```

NumPy提供了丰富的随机数生成函数，可以满足各种生成随机数数组的需求。在实际应用中，可以根据具体需求选择合适的函数，并通过指定参数来生成满足需求的随机数数组。